

La complexité algorithmique

expliquée à un débutant

De la recherche séquentielle au théorème maître : comprendre, mesurer et calculer la complexité des algorithmes itératifs et récur­sifs.

Sommaire

1. Un premier exemple : recherche séquentielle vs binaire	3
2. Qu'est-ce que la complexité ?	5
3. Calculer la complexité d'un programme itératif	7
4. Calculer la complexité d'un programme récur­sif	9
5. Le théorème maître	10
6. La méthode de substitution	12
7. Résumé et pièges à éviter	14

1. Un premier exemple : chercher un nom dans un annuaire

Avant de parler théorie, partons d'un exemple concret. Imagine un annuaire téléphonique papier trié par ordre alphabétique, contenant 1000 noms. Tu cherches un nom précis. Deux stratégies sont possibles.

1.1 Stratégie 1 : la recherche séquentielle

On part de la première page, et on regarde nom par nom jusqu'à trouver celui qu'on cherche (ou jusqu'à la fin si le nom n'existe pas). C'est simple, mais potentiellement très long.

```
def recherche_sequentielle(tableau, cible):  
    for i in range(len(tableau)):  
        if tableau[i] == cible:  
            return i  
    return -1 # non trouvé
```

Dans le pire des cas (le nom est en dernière position, ou n'existe pas), on doit examiner **les n éléments** du tableau. Le nombre d'opérations croît **proportionnellement** à n.

1.2 Stratégie 2 : la recherche binaire (dichotomie)

Puisque l'annuaire est trié, on peut faire beaucoup mieux : on ouvre le livre au milieu. Si le nom cherché vient avant alphabétiquement, on jette toute la seconde moitié et on recommence sur la première moitié — et inversement. À chaque étape, on **élimine la moitié des possibilités restantes**.

```
def recherche_binaire(tableau, cible):  
    gauche, droite = 0, len(tableau) - 1  
    while gauche <= droite:  
        milieu = (gauche + droite) // 2  
        if tableau[milieu] == cible:  
            return milieu  
        elif tableau[milieu] < cible:  
            gauche = milieu + 1  
        else:  
            droite = milieu - 1  
    return -1 # non trouvé
```

Ici, il faut que le tableau soit **déjà trié** — c'est la condition indispensable pour pouvoir éliminer une moitié à chaque étape.

1.3 Comparaison chiffrée

Voyons concrètement combien d'étapes sont nécessaires dans le pire des cas, selon la taille n de l'annuaire :

Taille n	Séquentielle (n)	Binaire ($\log_2 n$, arrondi)
10	10	4
1 000	1 000	10

Taille n	Séquentielle (n)	Binaire ($\log_2 n$, arrondi)
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

L'écart est spectaculaire : pour un milliard de noms, la recherche séquentielle peut demander un milliard de comparaisons, alors que la recherche binaire en demande à peine **30**. C'est précisément ce genre d'écart que la **théorie de la complexité** permet de prévoir et de comparer, sans avoir à tester le programme sur des exemples géants.

Idée clé à retenir : ce n'est pas la rapidité d'un ordinateur qui compte ici, mais la **stratégie** de l'algorithme. Un ordinateur 1000 fois plus rapide ne sauvera pas la recherche séquentielle face à la recherche binaire sur de très grands tableaux — l'écart se creuse quand n grandit.

2. Qu'est-ce que la complexité ?

La **complexité** d'un algorithme est une mesure de la quantité de ressources (temps de calcul, ou mémoire) qu'il consomme, exprimée **en fonction de la taille n de l'entrée**, et non en secondes ou en fonction d'un ordinateur particulier.

On distingue en général :

- **La complexité temporelle** : le nombre d'opérations élémentaires effectuées (c'est la plus étudiée).
- **La complexité spatiale** : la quantité de mémoire supplémentaire utilisée.

Dans ce document, on se concentre sur la complexité **temporelle**, et le plus souvent sur le **pire des cas** (worst case) : combien d'opérations au maximum, quelle que soit l'entrée de taille n .

2.1 Pourquoi ne pas simplement chronométrer le programme ?

- Le temps réel dépend de la machine, du langage, de la charge du processeur au moment du test...
- On veut une mesure **indépendante du matériel**, qui reste valable dans 5 ans sur un autre ordinateur.
- On veut surtout savoir comment le programme se comporte quand n devient **très grand** — c'est là que les mauvais algorithmes deviennent inutilisables.

2.2 Les notations asymptotiques

On utilise des notations qui décrivent le comportement d'une fonction quand n tend vers l'infini, en ignorant les constantes et les termes négligeables :

Notation	Signification	Exemple d'usage
$O(f(n))$	Borne supérieure : le programme ne fait jamais pire que $f(n)$ (à une constante près)	Recherche séquentielle : $O(n)$
$\Omega(f(n))$	Borne inférieure : le programme fait au moins $f(n)$ opérations	Trouver le min d'un tableau : $\Omega(n)$
$\Theta(f(n))$	Borne exacte : le programme se comporte exactement comme $f(n)$ (O et Ω à la fois)	Tri fusion : $\Theta(n \log n)$

Pour un débutant, l'essentiel est de bien maîtriser **$O()$** (le "pire cas", le plus utilisé en pratique) : on l'utilise pour dire "cet algorithme ne dépassera jamais, à peu de choses près, cet ordre de grandeur d'opérations".

2.3 Les ordres de grandeur courants, du meilleur au pire

Notation	Nom	Exemple typique
$O(1)$	Constante	Accéder à <code>tableau[i]</code>
$O(\log n)$	Logarithmique	Recherche binaire
$O(n)$	Linéaire	Recherche séquentielle, parcourir un tableau
$O(n \log n)$	Quasi-linéaire	Tri fusion, tri rapide (moyenne)

Notation	Nom	Exemple typique
$O(n^2)$	Quadratique	Boucles imbriquées, tri à bulles
$O(2^n)$	Exponentielle	Essayer tous les sous-ensembles
$O(n!)$	Factorielle	Essayer tous les ordres possibles

Retiens surtout l'**ordre** de cette liste : un algorithme en $O(n \log n)$ sera toujours, pour n assez grand, plus rapide qu'un algorithme en $O(n^2)$ — même si ce dernier a l'air "plus simple" à première vue.

3. Calculer la complexité d'un programme itératif

Pour un programme avec des boucles (for, while), la méthode générale consiste à compter combien de fois le "corps" de chaque boucle s'exécute.

3.1 Instructions simples : $O(1)$

Une affectation, une addition, une comparaison, un accès à un tableau par indice : chacune de ces opérations prend un temps **constant**, indépendant de n .

```
x = a + b
if x > 10:
    y = tableau[3]
```

→ Complexité : $O(1)$

3.2 Une boucle simple : $O(n)$

Si une boucle s'exécute n fois, et que chaque itération coûte $O(1)$, la complexité totale est $O(n)$.

```
somme = 0
for i in range(n):
    somme += tableau[i]
```

→ Complexité : $O(n)$ (n itérations $\times O(1)$ par itération)

3.3 Deux boucles l'une après l'autre : on additionne

```
for i in range(n):
    print(i)
for j in range(n):
    print(j)
```

→ Complexité : $O(n) + O(n) = O(2n) = O(n)$ (les constantes multiplicatives disparaissent dans la notation grand-O)

3.4 Boucles imbriquées : on multiplie

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Pour chacun des n tours de la boucle externe, la boucle interne fait n tours. →
Complexité : $n \times n = O(n^2)$

Règle pratique : **boucles l'une après l'autre** → on additionne. **Boucles imbriquées** → on multiplie.
C'est la règle la plus utile à retenir pour l'itératif.

3.5 Cas particulier : une boucle qui divise (comme la recherche binaire)

Reprenons la recherche binaire de la partie 1. À chaque tour de la boucle while, l'intervalle de recherche est **divisé par 2**. La question devient : combien de fois peut-on diviser n par 2 avant d'arriver à 1 ?

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$, soit k divisions où $n / 2^k = 1$, donc $k = \log_2 n$

$\rightarrow$ Complexité de la recherche binaire : $O(\log n)$

C'est exactement ce qu'on observait dans le tableau chiffré de la partie 1 : ~30 étapes pour un milliard d'éléments.

4. Calculer la complexité d'un programme récursif

Un programme récursif est un programme qui s'appelle lui-même sur une version plus petite du problème. Sa complexité ne se lit pas directement dans le code comme pour une boucle : on doit d'abord écrire une **relation de récurrence**, puis la résoudre.

4.1 Étape 1 : écrire la relation de récurrence

On note $T(n)$ le nombre d'opérations pour une entrée de taille n . On exprime $T(n)$ en fonction d'appels récursifs sur des tailles plus petites, plus le travail "local" fait à cet appel.

Exemple — recherche binaire écrite en version récursive :

```
def recherche_binaire_recursive(tableau, cible, gauche, droite):
    if gauche > droite:
        return -1
    milieu = (gauche + droite) // 2
    if tableau[milieu] == cible:
        return milieu
    elif tableau[milieu] < cible:
        return recherche_binaire_recursive(tableau, cible, milieu+1, droite)
    else:
        return recherche_binaire_recursive(tableau, cible, gauche, milieu-1)
```

À chaque appel : un travail constant $O(1)$ (calcul du milieu, comparaisons), puis **un seul** appel récursif sur une entrée de taille moitié. Cela se traduit par :

$$T(n) = T(n/2) + O(1)$$

Un autre exemple, le **tri fusion (merge sort)** : on divise le tableau en 2 moitiés, on trie chaque moitié récursivement, puis on fusionne (coût $O(n)$) :

$$T(n) = 2 \cdot T(n/2) + O(n)$$

4.2 Étape 2 : résoudre la relation de récurrence

Une fois $T(n)$ écrite, il reste à trouver quel est l'ordre de grandeur de $T(n)$. Trois méthodes principales existent, de la plus rapide à la plus générale :

Méthode	Principe	Quand l'utiliser
Arbre de récursion	Dessiner tous les niveaux d'appels et additionner le travail de chaque niveau	Pour comprendre / deviner la solution
Théorème maître	Formule toute faite pour $T(n) = a \cdot T(n/b) + f(n)$	Cas standards (tri, recherche...)
Substitution	Deviner une forme, puis la prouver par récurrence (induction)	Cas non standards, quand le théorème maître échoue

Voyons chacune de ces méthodes en détail, en reprenant nos deux exemples (recherche binaire et tri fusion).

4.3 L'arbre de récursion (pour bien visualiser)

L'idée : on représente chaque appel récursif comme un nœud d'un arbre. La racine est l'appel initial $T(n)$. Chaque nœud $T(k)$ a des enfants correspondant à ses appels récursifs. On additionne le travail fait à chaque **niveau** de l'arbre, puis sur tous les niveaux.

Exemple avec le tri fusion, $T(n) = 2 \cdot T(n/2) + O(n)$:

Niveau	Nombre de sous-problèmes	Taille de chacun	Travail total du niveau
0	1	n	n
1	2	$n/2$	$2 \times n/2 = n$
2	4	$n/4$	$4 \times n/4 = n$
...	...	...	...
$k = \log_2 n$	n	1	$n \times 1 = n$

Chaque niveau coûte n , et il y a $\log_2 n$ niveaux (l'arbre s'arrête quand la taille atteint 1). Total : $n \times \log_2 n$.

$$\rightarrow T(n) = \Theta(n \log n)$$

L'arbre de récursion est surtout un outil pour **deviner** la bonne réponse avant de la prouver proprement (avec le théorème maître ou la substitution).

5. Le théorème maître

Le théorème maître est un raccourci : pour toute relation de la forme suivante, il donne directement la réponse, sans refaire l'arbre à la main.

$$T(n) = a \cdot T(n/b) + f(n)$$

- **a** : le nombre d'appels récursifs faits à chaque étape
- **b** : le facteur de division de la taille (on résout un sous-problème de taille n/b)
- **f(n)** : le travail fait en dehors des appels récursifs (fusion, découpage, etc.)

5.1 D'où vient la comparaison avec $n^{\log_b a}$?

Cette quantité correspond au **nombre de feuilles** de l'arbre de récursion — c'est-à-dire le nombre de sous-problèmes de taille 1 générés au total.

Au niveau k , il y a a^k sous-problèmes, chacun de taille n/b^k . L'arbre s'arrête quand $n/b^k = 1$, c'est-à-dire quand $k = \log_b n$ (la **hauteur** de l'arbre). Le nombre de feuilles est donc :

$$a^{\log_b n} = n^{\log_b a} \text{ (identité algébrique classique)}$$

$n^{\log_b a}$ représente le coût "pur" de la récursion (si tout le travail se faisait aux feuilles), tandis que $f(n)$ représente le travail supplémentaire fait à chaque niveau. Le théorème maître compare ces deux quantités pour savoir laquelle domine.

5.2 Les trois cas

Cas	Condition	Résultat	Intuition
1	$f(n) = O(n^{\log_b a - \epsilon})$ (f croît plus lentement)	$T(n) = \Theta(n^{\log_b a})$	La récursion domine
2	$f(n) = \Theta(n^{\log_b a})$ (même ordre)	$T(n) = \Theta(n^{\log_b a} \cdot \log n)$	Équilibre parfait
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ + condition de régularité	$T(n) = \Theta(f(n))$	Le travail "extérieur" domine

5.3 La condition de régularité (cas 3 uniquement)

Pour le cas 3, il faut en plus que $f(n)$ vérifie :

$$a \cdot f(n/b) \leq c \cdot f(n), \text{ pour une constante } c < 1$$

Intuitivement, cela garantit que $f(n)$ ne "oscille" pas de façon pathologique d'un niveau à l'autre de l'arbre. En pratique, cette condition est **automatiquement vérifiée** pour presque toutes les fonctions usuelles (polynômes, log) — elle ne pose problème que pour des fonctions exotiques (avec des oscillations comme $n \cdot \sin n$).

5.4 Exemples résolus

Exemple A — Tri fusion $T(n) = 2 \cdot T(n/2) + O(n)$

$a = 2, b = 2 \rightarrow n^{\log_b a} = n^{\log_2 2} = n^1 = n$. $f(n) = n$, même ordre $\rightarrow$ **Cas 2**.

$$T(n) = \Theta(n \log n)$$

Exemple B — Recherche binaire $T(n) = T(n/2) + O(1)$

$a = 1, b = 2 \rightarrow n^{\log_b a} = n^0 = 1$. $f(n) = O(1)$, même ordre $\rightarrow$ **Cas 2**.

$$T(n) = \Theta(\log n)$$

Exemple C $T(n) = T(n/2) + O(n^2)$

$a = 1, b = 2 \rightarrow n^{\log_b a} = 1$. $f(n) = n^2$ croît beaucoup plus vite que 1.

Régularité : $a \cdot f(n/2) = (n/2)^2 = n^2/4 \leq (1/4) \cdot f(n)$ avec $c = 1/4 < 1$ ✓ $\rightarrow$ **Cas 3**.

$$T(n) = \Theta(n^2)$$

Piège fréquent : le théorème maître **ne s'applique pas toujours**. Il exige une différence **polynomiale** entre $f(n)$ et $n^{\log_b a}$ pour trancher entre les cas 2 et 3. Une fonction comme $f(n) = n \log n$ ("juste un peu" plus grande que n) tombe souvent entre deux chaises — c'est là qu'on utilise la méthode de substitution.

6. La méthode de substitution

Plus générale que le théorème maître : elle fonctionne même quand celui-ci échoue (fonctions non polynomiales, récurrences non standards...). Le principe : **deviner** la solution, puis la **prouver par récurrence** (induction mathématique).

- **Étape 1** : deviner une forme pour $T(n)$, souvent en s'aidant de l'arbre de récursion.
- **Étape 2** : prouver cette hypothèse par induction — on suppose qu'elle est vraie pour les valeurs plus petites, et on montre qu'elle reste vraie pour n .

6.1 Exemple classique — $T(n) = 2 \cdot T(n/2) + n$

Étape 1 (deviner) : on suppose $T(n) = O(n \log n)$, donc on va montrer $T(n) \leq c \cdot n \cdot \log n$ pour une constante $c > 0$ et n assez grand.

Étape 2 (prouver) : hypothèse d'induction sur $n/2$: $T(n/2) \leq c \cdot (n/2) \cdot \log(n/2)$.

$$\begin{aligned} T(n) &= 2 \cdot T(n/2) + n \\ &\leq 2 \cdot [c \cdot (n/2) \cdot \log(n/2)] + n \\ &= c \cdot n \cdot \log(n/2) + n \\ &= c \cdot n \cdot (\log n - \log 2) + n \\ &= c \cdot n \cdot \log n - c \cdot n \cdot \log 2 + n \end{aligned}$$

On veut que ce soit $\leq c \cdot n \cdot \log n$, donc il suffit que $-c \cdot n \cdot \log 2 + n \leq 0$, c'est-à-dire $c \geq 1/\log 2$. En choisissant c assez grand (par exemple $c = 2$), l'inégalité tient.

$$\rightarrow T(n) = O(n \log n) \checkmark$$

Le terme négatif " $-c \cdot n \cdot \log 2$ " vient absorber le "+ n " ajouté à chaque étape : c'est souvent tout l'enjeu d'une preuve par substitution.

6.2 Piège classique : une mauvaise hypothèse de départ

Pour ce même $T(n) = 2 \cdot T(n/2) + n$, essayons de deviner $T(n) = O(n)$:

$$T(n) \leq 2 \cdot c \cdot (n/2) + n = c \cdot n + n = (c + 1) \cdot n$$

Ce n'est **jamais** $\leq c \cdot n$, quelle que soit la constante c : il reste toujours un "+ n " en trop qu'on ne peut pas absorber. Cela prouve que $O(n)$ est une hypothèse trop optimiste — il fallait deviner plus large ($n \log n$).

6.3 Un cas où le théorème maître échoue : $T(n) = 2 \cdot T(n/2) + n \cdot \log n$

$a = 2$, $b = 2 \rightarrow n^{\log_b a} = n$. $f(n) = n \cdot \log n$ est plus grand que n , mais pas **polynomialement** plus grand (aucun $\varepsilon > 0$ ne vérifie $n \log n \geq n^{1+\varepsilon}$). Le théorème maître "basique" ne tranche pas ici.

Étape 1 (deviner) : on ajoute un facteur log en plus : $T(n) = O(n \cdot \log^2 n)$.

Étape 2 (prouver) : hypothèse d'induction : $T(n/2) \leq c \cdot (n/2) \cdot \log^2(n/2)$.

$$\begin{aligned} T(n) &\leq 2 \cdot c \cdot (n/2) \cdot \log^2(n/2) + n \cdot \log n \\ &= c \cdot n \cdot \log^2(n/2) + n \cdot \log n \end{aligned}$$

En utilisant $\log(n/2) = \log n - 1$, donc $\log^2(n/2) = \log^2 n - 2 \cdot \log n + 1$:

$$\begin{aligned} T(n) &\leq c \cdot n \cdot \log^2 n - 2c \cdot n \cdot \log n + c \cdot n + n \cdot \log n \\ &= c \cdot n \cdot \log^2 n - (2c - 1) \cdot n \cdot \log n + c \cdot n \end{aligned}$$

On veut que ce soit $\leq c \cdot n \cdot \log^2 n$, donc il suffit que $(2c - 1) \cdot \log n \geq c$. Pour n assez grand ($\log n$ grand), c'est vrai dès que $2c - 1 > 0$, soit $c > 1/2$. En prenant $c = 1$, l'inégalité tient pour $n \geq 2$.

$$\rightarrow T(n) = O(n \cdot \log^2 n) \checkmark$$

Ce que montre cet exemple : le théorème maître (version simple) gère mal les facteurs logarithmiques "en trop" dans $f(n)$, là où la substitution s'en sort très bien — il suffisait de deviner un log de plus.

6.4 Théorème maître vs substitution : que choisir ?

	Théorème maître	Méthode de substitution
Rapidité	Très rapide, presque automatique	Plus long, demande du calcul
Généralité	Limité à $T(n) = a \cdot T(n/b) + f(n)$ avec conditions	Marche pour presque toute récurrence
À utiliser pour	Cas standards (tri, recherche...)	Cas non standards, récurrences inhabituelles

7. Résumé et pièges à éviter

7.1 La méthode en un coup d'œil

- **Programme itératif** : boucles l'une après l'autre → on additionne. Boucles imbriquées → on multiplie. Une boucle qui divise la taille par 2 à chaque tour → $O(\log n)$.
- **Programme récursif** : écrire $T(n) = a \cdot T(n/b) + f(n)$, puis résoudre avec le théorème maître (rapide, cas standards) ou la substitution (générale, cas difficiles).
- **Toujours vérifier** a , b et $f(n)$ avant d'appliquer le théorème maître — une erreur fréquente est de se tromper sur qui est " a " et qui est " b ".

7.2 Pièges fréquents

- Confondre le **meilleur cas** et le **pire cas** : sauf indication contraire, on calcule presque toujours la complexité dans le pire des cas.
- Oublier la **condition de régularité** dans le cas 3 du théorème maître (rarement un problème en pratique, mais bon à savoir).
- Appliquer le théorème maître à une relation qui n'a pas exactement la forme $T(n) = a \cdot T(n/b) + f(n)$ (par exemple si les sous-problèmes n'ont pas tous la même taille).
- Deviner une hypothèse trop "serrée" en substitution (comme $O(n)$ au lieu de $O(n \log n)$) : si l'inégalité ne se referme jamais quelle que soit la constante choisie, c'est le signe qu'il faut deviner plus large.
- Confondre Θ (borne exacte) avec O (borne supérieure seulement) : dire " $T(n) = O(n^2)$ " pour un algorithme qui est en réalité $\Theta(n)$ n'est pas faux, mais c'est une information moins précise.

7.3 Pour aller plus loin

Une fois ces bases solides, les prochaines notions naturelles à explorer sont : la complexité **en moyenne** (average case, utile par exemple pour le tri rapide / quicksort), la complexité **amortie** (utile pour des structures comme les tableaux dynamiques), et la **complexité spatiale** (mémoire utilisée, avec les mêmes notations O , Ω , Θ).